

SAKTHIVEL BALAKUMAR – 63957

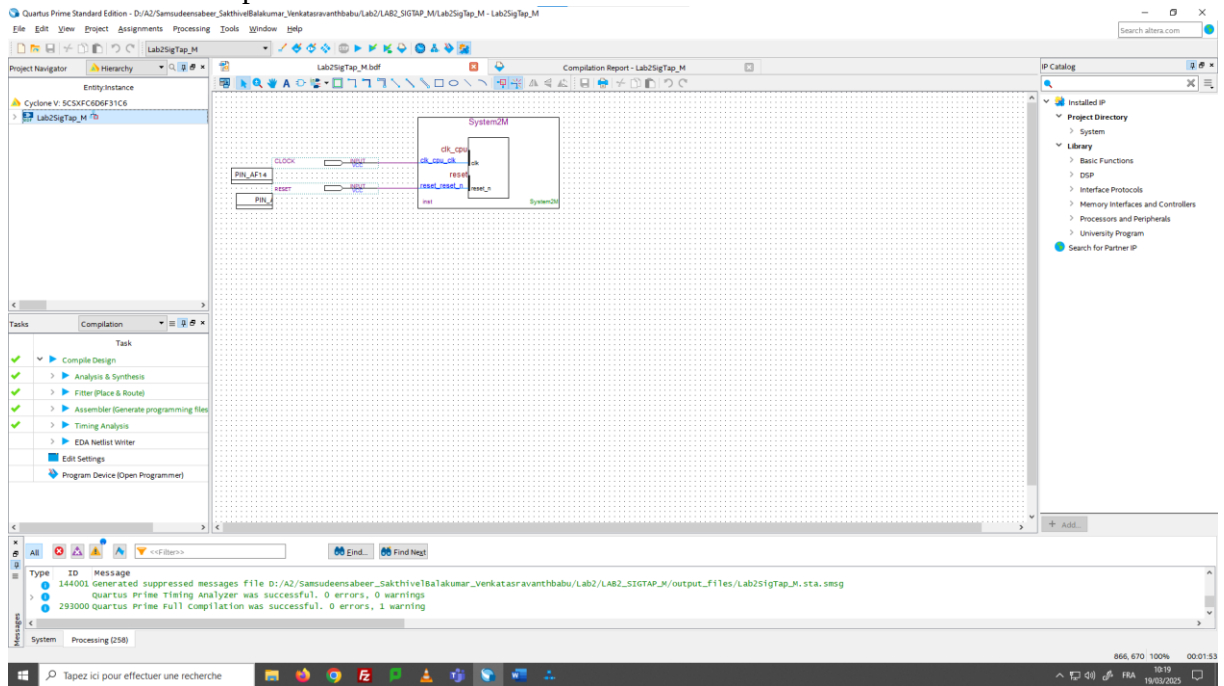
MEKA VENKATA SRAVANTH BABU – 63919

SAMSUDEEN SABEER RAHMAN – 63920

MICROSYSTEM ELECTRONICS - I.E.2409

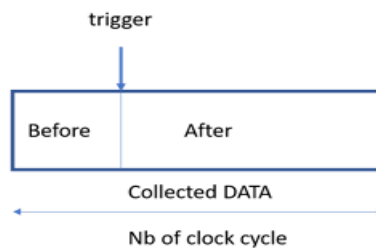
LAB 2

- Integrate Signal TAP
- How to add a component on SoC



Integrate Signal Tap – Avalon Bus

During this lab we have observed and understood the signals of the bus used to exchange data between the processor and its peripherals. In this particular case, the processor is the NIOS2 and bus is AVALON. We have used a Logic Analyzer which permits to see all the desired logic signal. (The values upon the time). In the FPGA SignalTap is an IP of logic analyzer handle by Quartus. Signal Tap can be incorporated with a design inside the FPGA and permits to see any logical signals inside the FPGA and to catch some part of it depending some specific conditions. The parametrization permits to define the signals we want to see.



The parameters are:

- **The clock:** at every edge of this signal, the signals will be sampled and stored
- **Some signals inside the fpga:** which could be internal signal in a design (as example Avalon bus) but also some signals on I/O or signal related to a specific IP.
- **The number of collected Data:** with an impact of the memory in the fpga
- **The trigger conditions:** which can simple: an edge on a specific signal or more complex.

EXPERIMENT

The purpose of the following is to see and to understand signals on Avalon Bus.

Q1) Enumerate the elements in this system?

Ans:

CLK_CPU, NIOS2_CPU, Jtag_uart_0, Onchip_Mem and TST_IP

Definition :

CLK_CPU is the clock signal used by the system. **NIOS2_CPU** is the softcore processor implemented inside the FPGA. **Jtag_uart_0** is used for communication between NIOSII processor and a computer. **Onchip_Mem** is used for storing data, instructions, or temporary variables.

Name	Description	Export	Clock	Base	End	IRQ	Tags	Opcode Name
CLK_CPU	Clock Source	clk_cpu reset	exported					
clk_in	Clock Input	Double-click to export	CLK_CPU					
clk_in_reset	Reset Input	Double-click to export						
clk	Clock Output	Double-click to export						
clk_reset	Reset Output	Double-click to export						
NIOS2_CPU	Noe II Processor							
clk	Clock Input	Double-click to export	CLK_CPU					
reset	Reset Input	Double-click to export						
data_master	Avalon Memory Mapped Master	Double-click to export						
instruction_master	Avalon Memory Mapped Master	Double-click to export						
irq	Interrupt Receiver	Double-click to export				IRQ 0	IRQ 31	
debug_reset_request	Reset Output	Double-click to export						
debug_mem_slave	Avalon Memory Mapped Slave	Double-click to export		0x0004_0000	0x0004_ffff			
custom_instruction_m...	Custom Instruction Master	Double-click to export						
Jtag_uart_0	JTAG UART Intel FPGA IP							
clk	Clock Input	Double-click to export	CLK_CPU					
reset	Reset Input	Double-click to export						
avalon_jtag_slave	Avalon Memory Mapped Slave	Double-click to export		0x0004_1400	0x0004_1407			
irq	Interrupt Sender	Double-click to export						
Onchip_Mem	On-Chip Memory (RAM or ROM) Intel ...							
clk1	Clock Input	Double-click to export	CLK_CPU					
s1	Avalon Memory Mapped Slave	Double-click to export		0x0002_0000	0x0003_ffff			
reset1	Reset Input	Double-click to export						
TST_IP	IP_Mem							
clk	Clock Input	Double-click to export	CLK_CPU					
rst	Reset Input	Double-click to export						
slave	Avalon Memory Mapped Slave	Double-click to export		0x0008_0000	0x0008_03ff			

Q2) What is the memory size?

Ans : The memory size is **131072 bytes**. The memory size here refers to a 512 KB memory block that is configured as RAM (writable) with dual-port access and a 32-bit data width.

System: System2M Path: OnChip_Mem

On-Chip Memory (RAM or ROM) Intel FPGA IP
altera_avalon_onchip_memory2

Memory type
Type: RAM (Writable)
☐ Dual-port access
☐ Single clock operation
Read During Write Mode: DONT_CARE
Block type: AUTO

Size
☐ Enable different width for Dual-port access
Slave S1 Data width: 32
Total memory size: 131072 bytes
☐ Minimize memory block usage (may impact tmax)

Read latency
Slave s1 Latency: 1
Slave s2 Latency: 1

Details
System Contents Address Map Interconnect Requirements
System: System2M Path: OnChip_Mem

Connections

Use	Connections	Name
☒		CLK_CPU
☒		clk_in
☒		clk_in_reset
☒		clk
☒		clk_reset
☒		NIOS2_CPU
☒		clk
☒		reset
☒		data_master
☒		instruction_master
☒		irq
☒		debug_reset_request
☒		debug_mem_slave
☒		custom_instruction_m...
☒		jtag_uart_0
☒		clk
☒		reset
☒		avalon_jtag_slave
☒		irq
☒		OnChip_Mem

Q3) What is the address and the size (number of reserved memory cells) of the component "TST_IP" an instance of "IP_Mem" IP ?

Ans: Base 0x0008_0000 and End 0x0008_03ff

Explanation: The Base address is the starting address and End address is the last address of the reserved memory range. The reserved memory block spans **1024** memory locations. If each memory location corresponds to a byte, this block would be **1KB** in size.

Platform Designer - System2M.qsys (D:\A2\Samsudeensabeer_SakthivelBalakumar_Venkatasravanthbabu\Lab2\LAB2_SIGTAP_M\System2M.qsys)
File Edit System Generate View Tools Help

System: System2M Path: CLK_CPU

Connections	Name	Description	Export	Clock	Base	End	IRQ	Tags	Opcode Name
☒	CLK_CPU	Clock Source	clk_cpu	exported					
☒	clk_in	Clock Input	clk_in_reset	CLK_CPU					
☒	clk_in_reset	Reset Input	clk_in_reset	CLK_CPU					
☒	clk	Clock Output	clk	CLK_CPU					
☒	clk_reset	Reset Output	clk_reset	CLK_CPU					
☒	NIOS2_CPU	Nios II Processor	Double-click to export	CLK_CPU					
☒	reset	Reset Input	Double-click to export	CLK_CPU					
☒	data_master	Avalon Memory Mapped Master	Double-click to export	CLK_CPU					
☒	instruction_master	Avalon Memory Mapped Master	Double-click to export	CLK_CPU					
☒	irq	Interrupt Receiver	Double-click to export	CLK_CPU					
☒	debug_reset_request	Reset Output	Double-click to export	CLK_CPU					
☒	debug_mem_slave	Avalon Memory Mapped Slave	Double-click to export	CLK_CPU	0x0004_0800	0x0004_0fff			
☒	custom_instruction_m...	Custom Instruction Master	Double-click to export	CLK_CPU					
☒	jtag_uart_0	JTAG UART Intel FPGA IP	Double-click to export	CLK_CPU					
☒	clk	Clock Input	Double-click to export	CLK_CPU					
☒	reset	Reset Input	Double-click to export	CLK_CPU					
☒	avalon_jtag_slave	Avalon Memory Mapped Slave	Double-click to export	CLK_CPU	0x0004_1400	0x0004_1407			
☒	rst	Interrupt Sender	Double-click to export	CLK_CPU					
☒	OnChip_Mem	On-Chip Memory (RAM or ROM) Intel ...	Double-click to export	CLK_CPU					
☒	s1	Clock Input	Double-click to export	CLK_CPU					
☒	reset1	Reset Input	Double-click to export	CLK_CPU	0x0002_0000	0x0002_ffff			
☒	IP_Mem	IP Mem	Double-click to export	CLK_CPU					
☒	clk	Clock Input	Double-click to export	CLK_CPU					
☒	rst	Reset Input	Double-click to export	CLK_CPU	0x0008_0000	0x0008_03ff			
☒	slave	Avalon Memory Mapped Slave	Double-click to export	CLK_CPU					

Current filter:

Messages

Type	Path	Message
Info	System2M_jtag_uart_0	JTAG UART IP input clock need to be at least double (2x) the operating frequency of JTAG TCK on board

0 Errors, 0 Warnings

Generate HDL... Finish

Read waveforms and write waveforms :

The screenshot shows the Platform Designer - System2M.qsys interface. On the left, the 'Parameters' tab for 'Avalon Memory Mapped Slave' is visible, showing settings for address units, associated clock, reset, and burst count. The 'Timing' section shows read and write wait times. The 'Read Waveforms' and 'Write Waveforms' sections show timing diagrams for clock, read/write, chipselect, address, and read/write data.

The main window displays the 'System Contents' table, which lists the components and their connections. The table has columns for Name, Description, Export, Clock, Base, End, IRQ, Tags, and Opcode Name.

Name	Description	Export	Clock	Base	End	IRQ	Tags	Opcode Name
CLK_CPU	Clock Source	clk_cpu	exported					
clk_in_reset	Reset Input	Double-click to export						
clk_reset	Reset Output	Double-click to export	CLK_CPU					
NIOS2_CPU	Nios II Processor	Double-click to export						
clk	Clock Input	Double-click to export	CLK_CPU					
reset	Reset Input	Double-click to export	[ck]					
data_master	Avalon Memory Mapped Master	Double-click to export	[ck]					
instruction_master	Avalon Memory Mapped Master	Double-click to export	[ck]					
irq	Interrupt Receiver	Double-click to export	[ck]					
debug_reset_request	Reset Output	Double-click to export	[ck]					
debug_mem_slave	Avalon Memory Mapped Slave	Double-click to export	[ck]	0xa0004_0000	0xa0004_ffff	IRQ 0	IRQ 31	
custom_instruction_m...	Custom Instruction M...	Double-click to export	[ck]					
Jtag_uart_0	JTAG UART Intel FPGA IP	Double-click to export						
clk	Clock Input	Double-click to export	CLK_CPU					
reset	Reset Input	Double-click to export	[ck]					
avalon_jtag_slave	Avalon Memory Mapped Slave	Double-click to export	[ck]	0xa0004_1400	0xa0004_1407			
irq	Interrupt Sender	Double-click to export	[ck]					
OnChip_Mem	On-Chip Memory (SRAM or ROM) Intel ...	Double-click to export						
clk1	Clock Input	Double-click to export	CLK_CPU					
s1	Avalon Memory Mapped Slave	Double-click to export	[ck]	0xa0002_0000	0xa0003_ffff			
reset1	Reset Input	Double-click to export	[ck]					
TST_IP	IP_Nmem	Double-click to export	CLK_CPU					
clk	Clock Input	Double-click to export	[ck]					
rst	Reset Input	Double-click to export	[ck]					
slave	Avalon Memory Mapped Slave	tst_ip_slave	[ck]					
IP_Nmem_0	IP_Nmem	unconnected #						

Again returned to QUARTUS Software, the project is compiled.

The DE10 board is programmed .

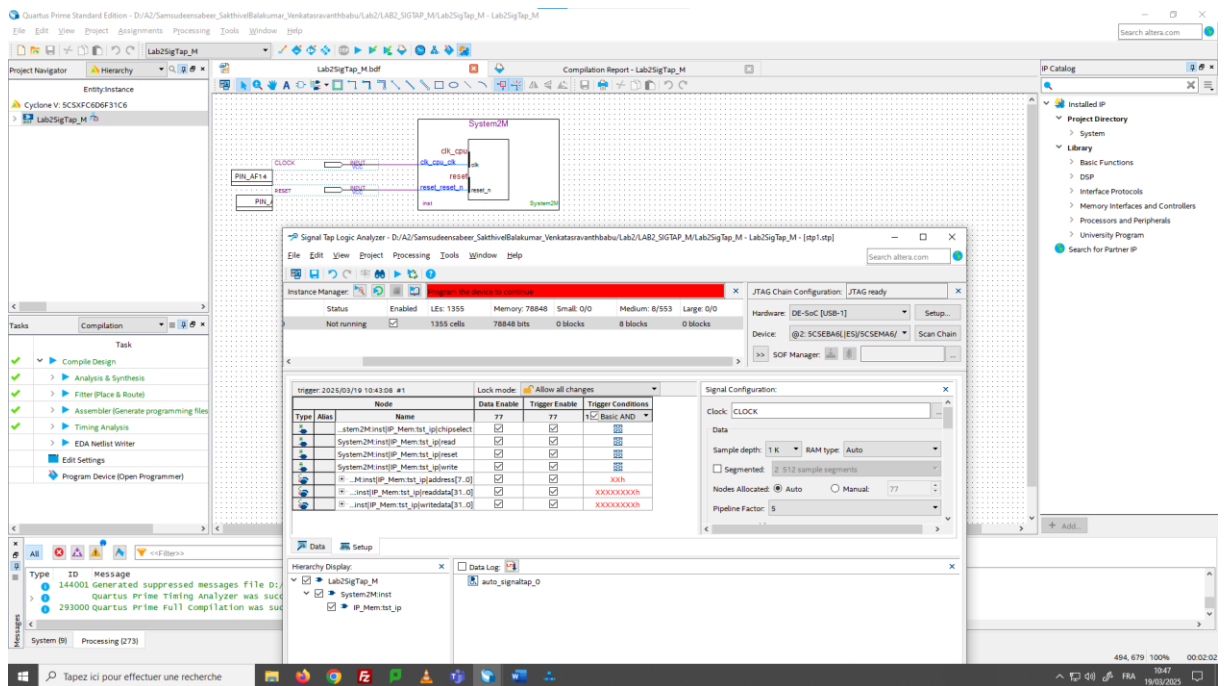
Now we integrate a SIGNAL TAP to view the signals around the component TST_IP.

The clock is selected as follow:

- 1) The filter is selected to "Pins: all" and click on "List":
- 2) The signal « CLOCK » is selected and pressed OK.

The signals are then added.

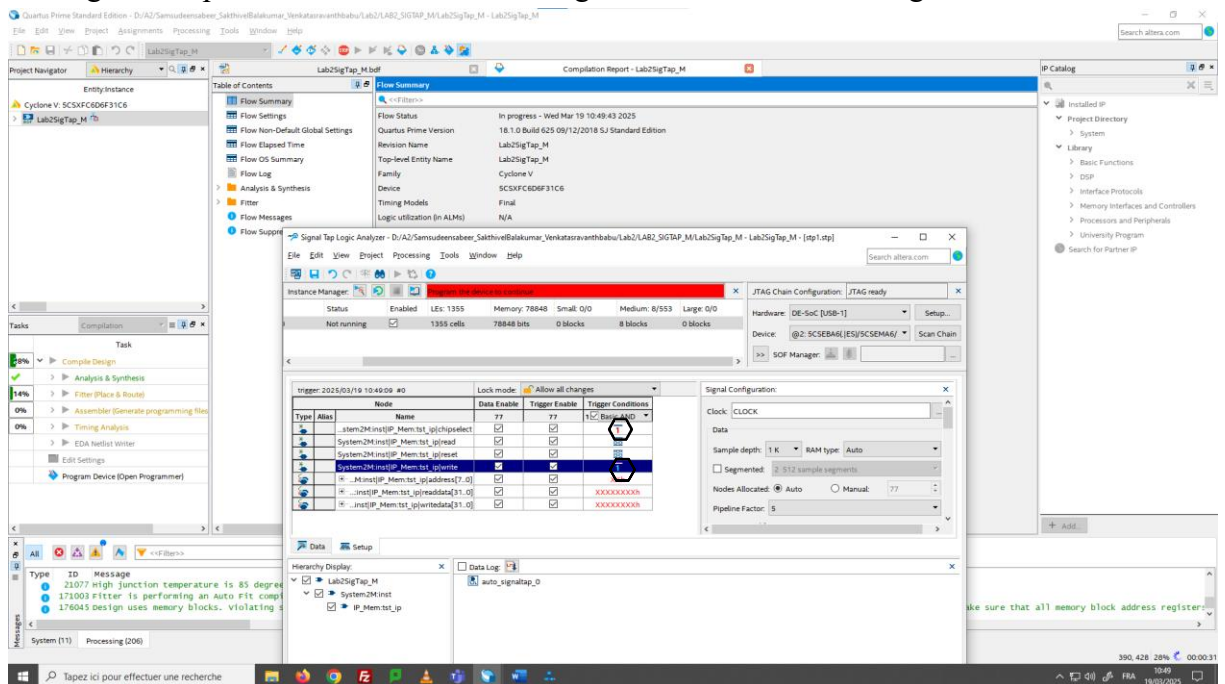
The filter "Design Entry (all names)" is selected and click on "List". Then the AVALON BUS signals are selected. "Insert" is clicked and the windows are closed.



The file is saved under the proposed name.

Now we are choosing a trigger:

The signal “chipselect” is selected and configured it to wait a HIGH signal on it.

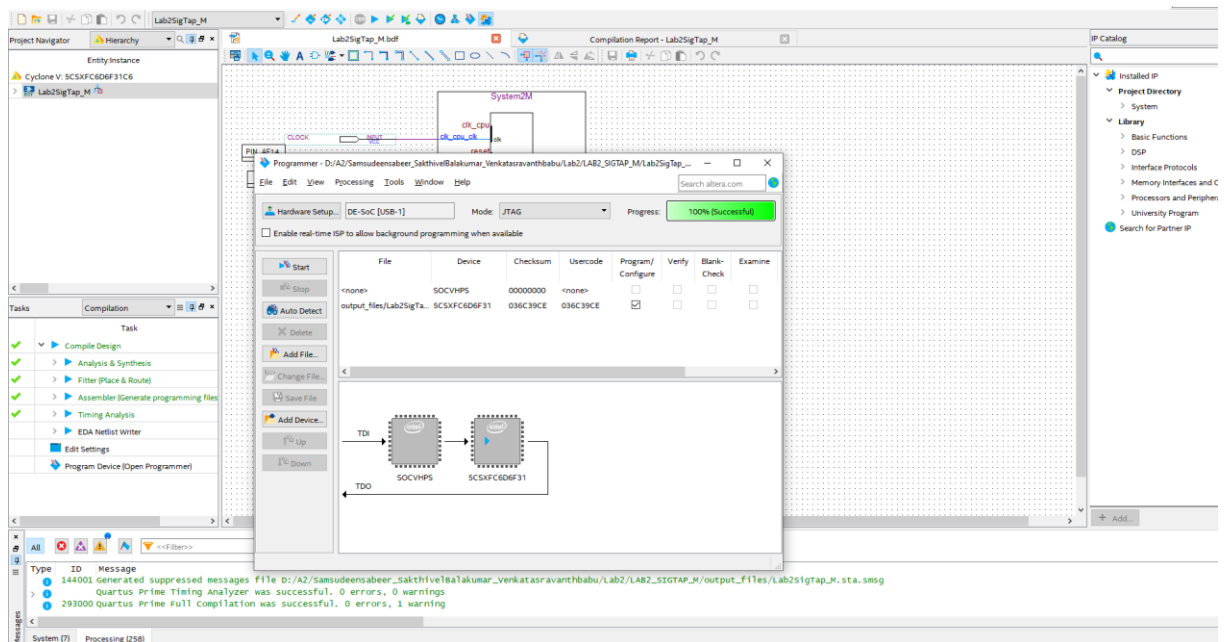


The file is saved.

The compilation is ran.

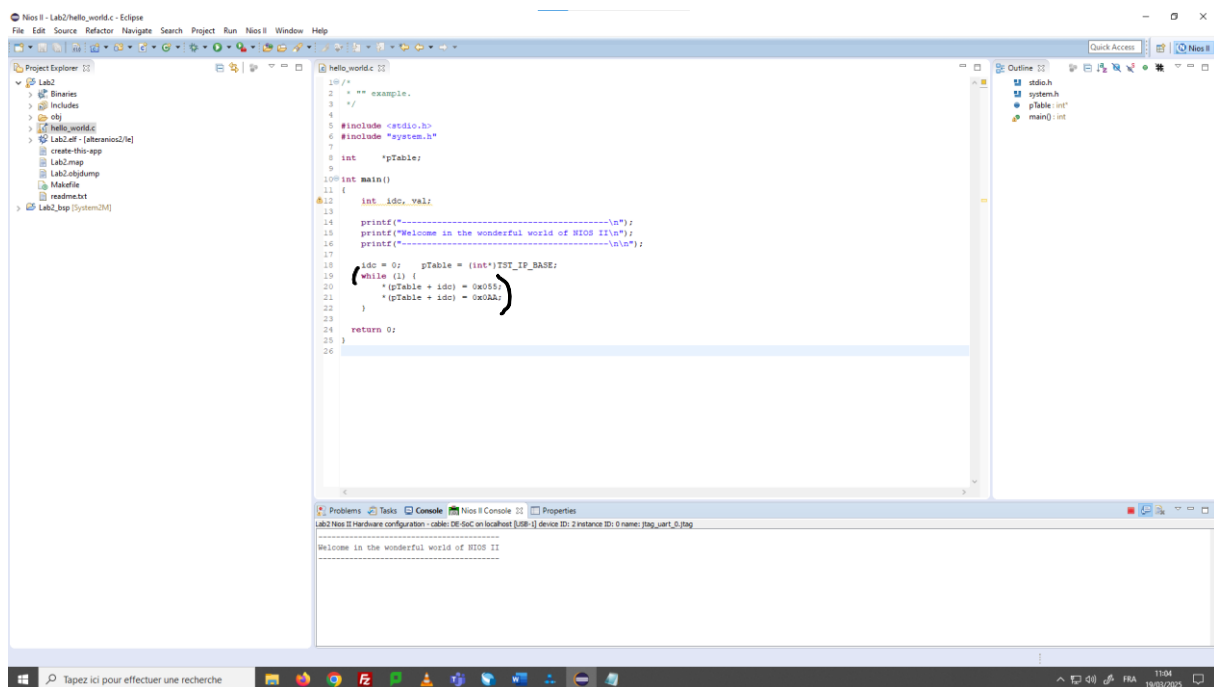
The board is programmed again.

A new project is created, we copied the code provided on the file “Lab2_IPMem.c”.



Q4) Explain the content of the while(1) loop

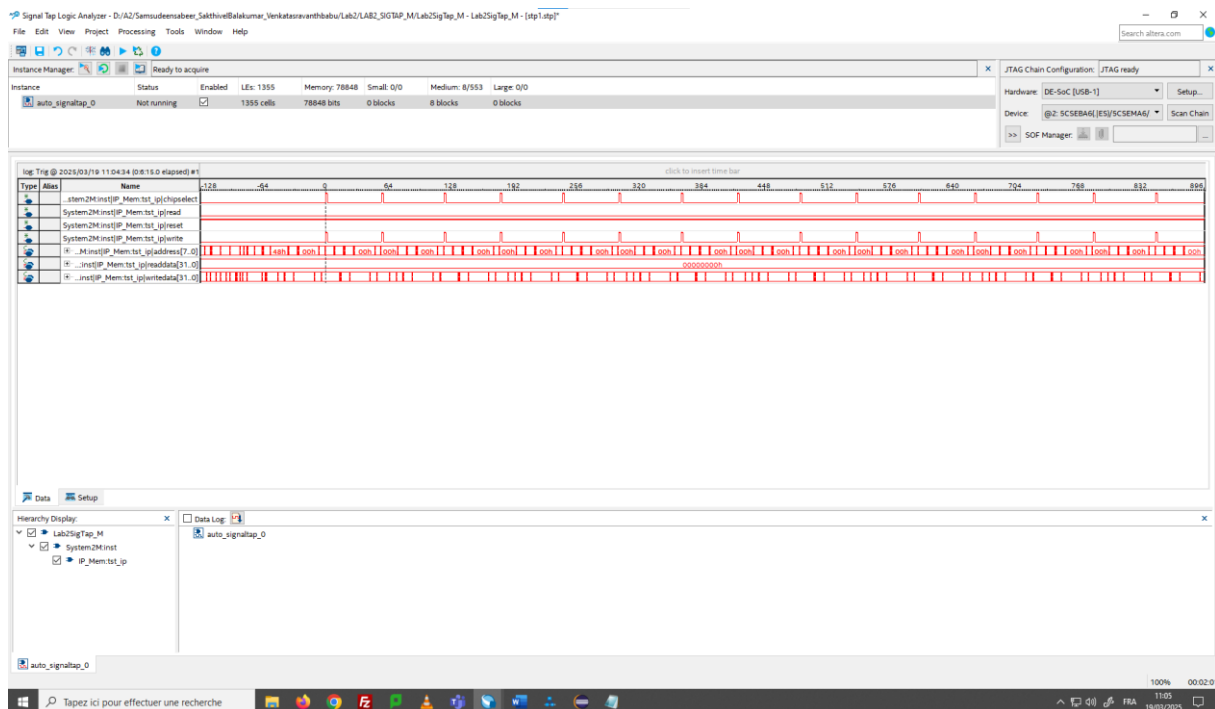
Ans :



Explanation: The while(1) loop in the given c program is an infinite loop.

pTable is a pointer to an integer, initialized to TST_IP_BASE, which is nothing but a memory mapped hardware address. idc is initialized to 0, that is pTable + idc points to the same address as pTable. The loop continuously writes 0x55 followed by 0xAA to the same memory location. It writes the hexadecimal value 0x55 to the memory location pointed to by pTable. Then next it overwrites the same memory location with 0xAA. Now the program is ran.

The AVALON BUS signals the generated.



Q5) Explain the signals

Ans:

The signals shown belong to a memory interface system. This waveform represents various control and data signals related to memory transactions. The patterns that are repeating in the waveforms explain the periodic memory transactions occurring in a synchronous system.

- The Mem_st.iplsreset signal is used as a reset for the memory.
- The Mem_st.iplswritedata and Mem_st.iplswrite signals represent us that data is written to memory.
- The Mem_st.iplsread and Mem_st.iplsreaddata signals handle the memory read operations.

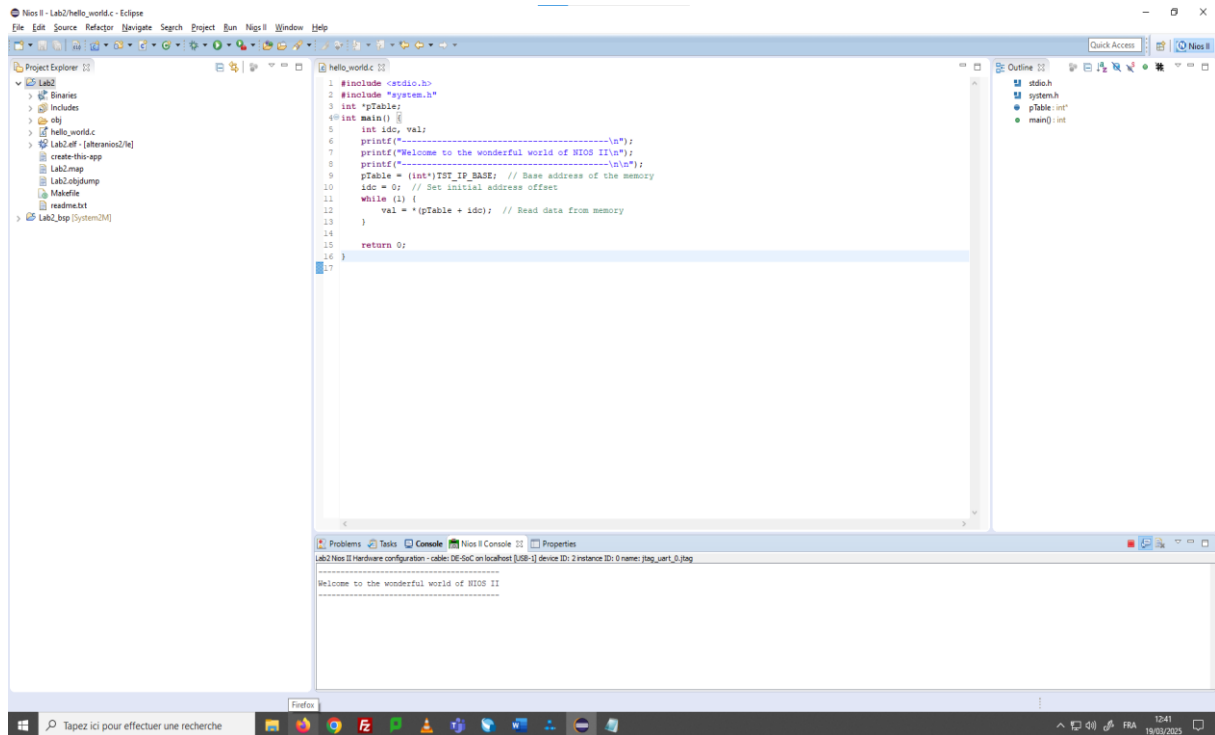
Q6) Write a program to perform a READ operation and display the chronograms.

Ans: Program:

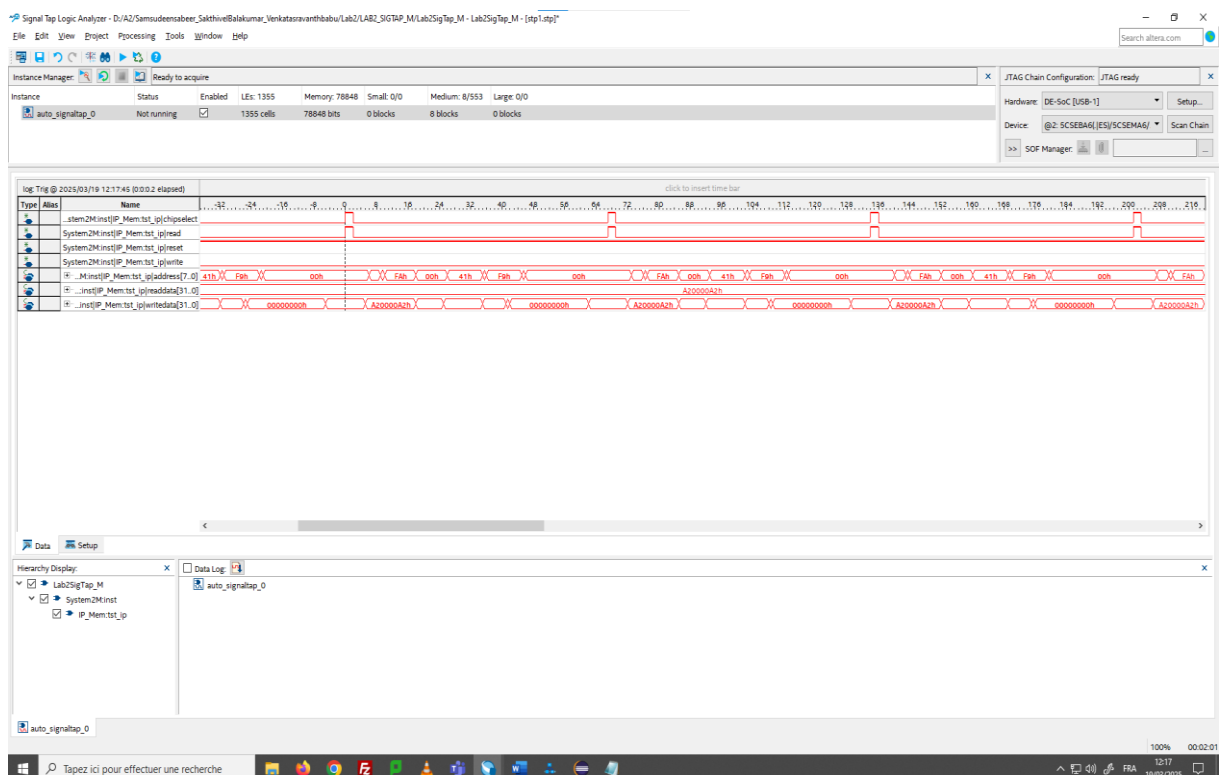
```
#include <stdio.h>
#include "system.h"
int *pTable;
int main() {
    int idc, val;
    printf("-----\n");
    printf("Welcome to the wonderful world of NIOS II\n");
    printf("-----\n\n");
    pTable = (int*)TST_IP_BASE; // Base address of the memory
    idc = 0; // Set initial address offset
    while (1) {
        val = *(pTable + idc); // Read data from memory
    }
    return 0;
}
```


Explanation : This program is a simple NIOSII application that reads data from a memory-mapped location. It initializes a pointer **pTable** to a base address defined by **TST_IP_BASE**, which is typically a memory-mapped address of a hardware component. After entering into infinite loop it continuously reads values from memory at an offset **idx** from the base address.

Output:



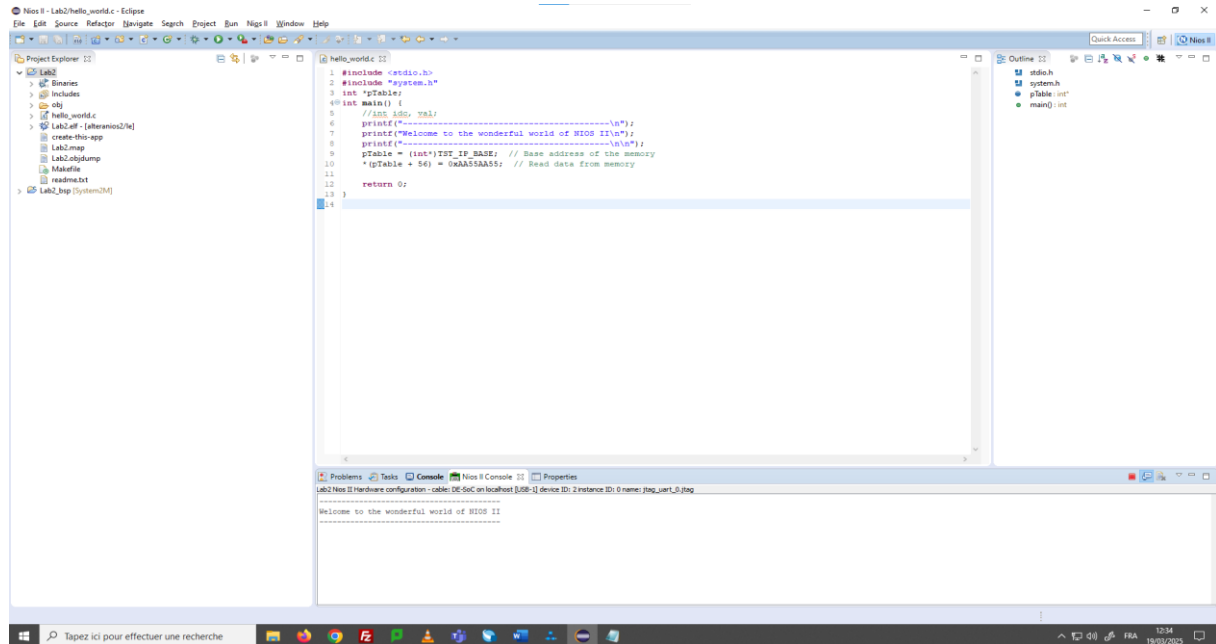
Waveform :



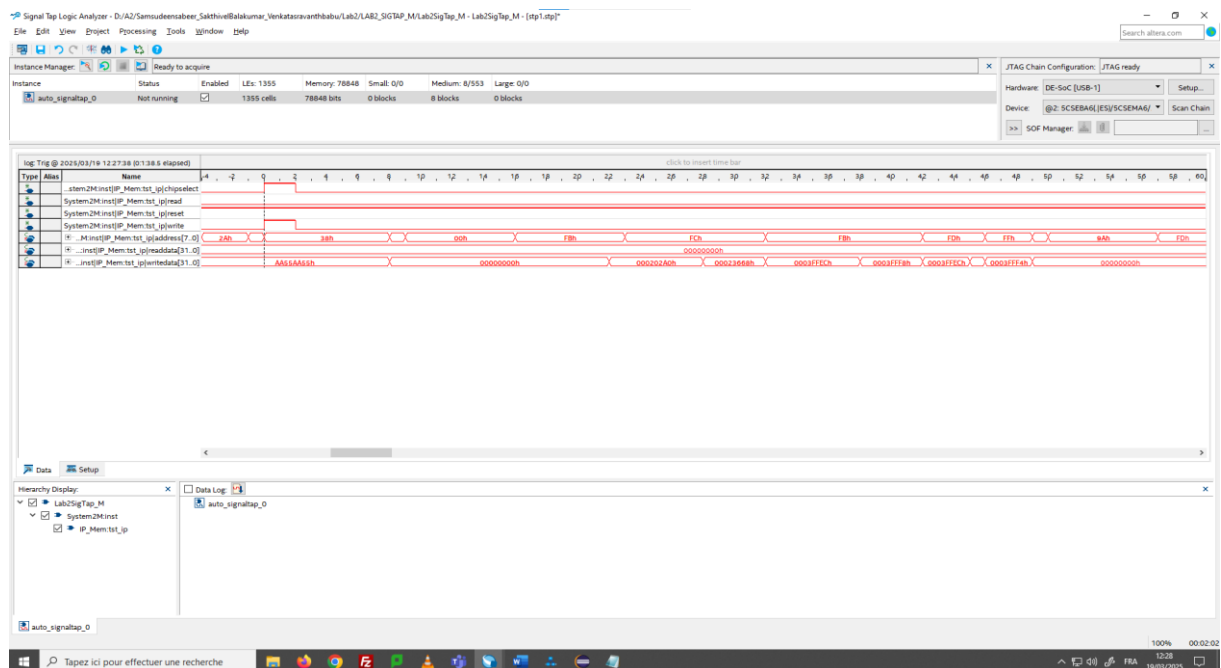
Q7) Modify the program to perform a write 0xAA55AA55 at the address 56.

Ans :

Output :



Waveform :



Program :

```
#include <stdio.h>
#include "system.h"
int *pTable;
int main() {
    //int idc, val;
```

```

printf("-----\n");
printf("Welcome to the wonderful world of NIOS II\n");
printf("-----\n\n");
pTable = (int*)TST_IP_BASE; // Base address of the memory
*(pTable + 56) = 0xAA55AA55; // Read data from memory

return 0;
}

```

Explanation : This program writes a specific value to a memory-mapped location. It initializes a pointer **pTable** to a base address defined by TST_IP_BASE, which typically corresponds to a hardware component's memory address. The program then writes the value **0xAA55AA55** to an offset of 56 from this base address.

Q8) Add a loop in the program to write values from N to N+255 at successive addresses.

Ans :

Output :

```

1 #include <stdio.h>
2 #include "system.h"
3 int *pTable;
4 int main() {
5     int N = 1; // Starting address oAset
6     int i;
7     printf("-----\n");
8     printf("Welcome to the wonderful world of NIOS II\n");
9     printf("-----\n\n");
10    pTable = (int*)TST_IP_BASE; // Base address of the memory
11    // Loop to write values from N to N+255 at successive addresses
12    while(i){
13        for (i = 0; i <= 255; i++) {
14            *(pTable + (N + i)) = 0xAA55AA55;
15        }
16    }
17    return 0;
18 }
19

```

Lab2: Nios II Hardware configuration - cable: DE-64C on localhost [JTAG-0] device ID: 2 instance ID: 0 name: jtag_uart_0.jtag

Welcome to the wonderful world of NIOS II

0xAA55AA55

Program :

```

#include <stdio.h>
#include "system.h"
int *pTable;
int main() {
    int N = 1; // Starting address oAset
    int i;
    printf("-----\n");
    printf("Welcome to the wonderful world of NIOS II\n");
    printf("-----\n\n");
}

```

```

pTable = (int*)TST_IP_BASE; // Base address of the memory
// Loop to write values from N to N+255 at successive addresses
while(1){
    for (i = 0; i <= 255; i++) {
        *(pTable + (N + i)) = N + i;
    }
}
return 0;
}

```

Explanation : This program continuously writes a sequence of values to a memory-mapped location. It initializes a pointer **pTable** to a base address defined by **TST_IP_BASE** and uses a loop to write values from **N** to **N+255** at successive memory addresses. The program runs indefinitely, repeatedly updating the memory with these values.

Waveform :



Q9) Reading the signals with SignalTap what is the time between two successive writes?

Ans :STEPS :

- Setup SignalTap
- Trigger the configuration
- Capture data
- Analyze the Captured Data
- To calculate the time between the writes the following formula can be used:

Time between writes = (Number of clock cycles between writes) × (Clock period)

- The clock period is the inverse of the clock frequency:

$$\text{Clock period} = \frac{1}{\text{Clock frequency}}$$